COMPTIA PENTEST+ · PT0-003 · DOMAIN 4.0 ATTACKS & EXPLOITS

4.6 — Cloud-Based Attacks

The cloud domain rewards **pattern recognition**, not memorized commands. Almost every question is a scenario that hands you a tell — an IP, a permission string, a config flag, a tool name — and asks you to name the attack, pick the tool, or predict the next hop. This guide gives you a map to slot each tell into, the analogies to keep them straight, and the confusable pairs the exam loves to blur.

10 ATTACK TYPES | 6 TOOL FAMILIES | 3 ATTACK CHAINS | CONFUSABLES CRAM | SCENARIO RED-FLAGS DECODER

HOW TO READ THIS GUIDE

Picture it
An analogy that makes the mechanism stick so you recognize it under exam pressure.

Exam tell
The exact words/values a scenario drops that point straight at this answer.

Planes
Every attack is filed under one of 5 “planes” — the map below is your mental filing cabinet.

THE SCAFFOLDING — 5 CLOUD ATTACK PLANES

Cloud attacks feel like a pile of unrelated tricks until you sort them by **where the attack lives**. Everything in 4.6 drops into one of five planes. When a scenario appears, first ask “*which plane?*” — that alone eliminates most wrong answers.

FILE EVERY 4.6 SCENARIO INTO ONE PLANE

① IDENTITY PLANE
Who can do what. The fastest path to takeover.
- IAM misconfiguration • Trust relationship abuse
- Metadata service → temporary credentials (SSRF)

Tell: role, policy, "*", assume-role, 169.254.169.254

② NETWORK PLANE
What can reach what.
- Network segmentation failure (flat network)
- Network controls • Public access to services

Tell: security group, 0.0.0.0/0, VPC, public endpoint

③ DATA / STORAGE PLANE
Where the secrets sit.
- Exposed storage buckets • Logging info exposure

Tell: public bucket, S3 ACL, CloudTrail logs, verbose logs

④ SUPPLY / ARTIFACT PLANE
What gets deployed — before it runs.
- Supply chain • Image/artifact tampering • 3rd-party integ.

Tell: registry, AMI, dependency, pipeline, typosquat, webhook

⑤ RUNTIME PLANE
The workload while it executes — and breaking out of it.
- Workload runtime attacks (app vuln → code exec inside the workload)
- Container escape (break the isolation boundary → reach the host)

Tell: privileged container, /var/run/docker.sock, hostPID, Lambda, code execution, namespace

Chains flow across planes: ⑤ runtime code-exec → ① grab creds → ② move → ③ loot. The exam tests the hops.

The single most-tested chain lives in the Identity plane → see the crown-jewel box below.

👑 The crown-jewel chain: SSRF → Metadata → IAM credentials

This is the single most-tested idea in cloud attacks. If you memorize one chain, memorize this one.

SSRF in a web app (runtime plane) →

server fetches `http://169.254.169.254/latest/meta-data/…` →

returns **temporary IAM creds** for the instance role → attacker calls AWS APIs with the role's permissions

Why it bypasses the firewall: the request originates from the *trusted instance itself* — the metadata IP `169.254.169.254` is a **link-local** address, only routable from inside the box, so no perimeter control ever sees an "external" hit. **IMDSv1** = a simple `GET` (easy to abuse). **IMDSv2** = you must `PUT` for a session token first (harder, not impossible). Bonus loot: the **user-data** field often holds bootstrap scripts stuffed with hardcoded secrets.

PART ONE — ATTACK TYPES (BY PLANE)

① Identity plane

1 Metadata service attacks

SSRF an app on a cloud instance into querying the internal metadata endpoint, which hands back the instance's temporary IAM credentials (and user-data secrets).

PICTURE IT A hotel room phone that dials the front desk and the desk reads out the room's master keycard — but only if you're calling *from inside the room*. SSRF is tricking the app into making that call for you.

EXAM TELL `169.254.169.254` in a scenario, or "SSRF retrieves temporary credentials," or IMDSv1 vs v2 → **metadata attack**.

2 IAM misconfiguration

Identities carry more permission than they should. This is the **most direct path to privilege escalation** in the cloud — IAM authorizes every action, so any weakness has unlimited blast radius.

PICTURE IT An office where every intern's badge happens to open the vault. Nothing was "hacked" — the badge was just over-scoped.

FLAVORS Wildcard policies (`"Action":"*"` / `"Resource":"*"`) · over-scoped instance/service roles · stale keys · missing MFA · misconfigured federation · **exposed access keys in code repos** (harvested within minutes) · overly broad cross-account trust.

ESCALATION If a low-priv user can attach policies to itself, create users, or assume a higher role → they climb to admin.

EXAM TELL "user has more permissions than expected," "key found in a repository," wildcard permission strings, least-privilege violation → **IAM misconfiguration**. Tools named: **Prowler** **ScoutSuite** **Pacu**.

10 Trust relationship abuse

Cloud runs on trust links (cross-account roles, federation, service-to-service, OAuth). Abuse them to **move laterally and escalate across boundaries** without exploiting each target directly.

PICTURE IT — A backstage lanyard that every venue in the tour agrees to honor. Compromise one venue's lanyard and you walk into all of them.

FLAVORS — Overly broad **cross-account trust** policies · **Kerberoasting** & unconstrained delegation in hybrid AD · **federated identity** (SAML) abuse · **OAuth** scope/token abuse · service-to-service over-trust.

EXAM TELL — Attacker "moves from one account/service to another," SPN/Kerberoasting, SAML, cross-account role, OAuth scope → **trust relationship abuse**. Tool: BloodHound maps the trust graph.

② Network plane · ③ Data plane — Resource misconfiguration

The exam breaks "resource misconfiguration" into five named sub-parts. Learn them by the **distinct attack path each one opens**.

SUB-PART	WHAT BREAKS	EXAM KEYWORDS
4a · Segmentation Network	No isolation → **flat network**. One compromised box can reach DBs, internal APIs, mgmt interfaces without any pivot. Turns a limited compromise into a full one.	VPC, subnet, flat network, private subnet, lateral movement, NACLs
4b · Network controls Network	Over-permissive security groups / NACLs. Internal services become directly reachable from the internet; missing egress filtering lets C2 & exfil out.	security group, NACL, `0.0.0.0/0`, open ports (22/3389/3306/5432/1433), egress filtering
4c · IAM credentials Identity	Over-permissioned **role attached to the resource**. Compromise the box → inherit the role. If the role is admin, you're admin.	instance role, service account, over-permissioned, inherited permissions
4d · Exposed buckets Data	Public read on object storage → **unauthenticated** access to backups, config, logs, source, secrets. No exploit needed — just the bucket name.	S3 bucket, public access, ACL, bucket policy, `aws s3 ls`, data exposure
4e · Public services Network	Things that should be internal are internet-facing: **Kubernetes API**, databases (RDS/Cloud SQL), registries, dashboards (Grafana, K8s Dashboard, Elasticsearch).	public endpoint, Kubernetes API, database exposure, admin console, internet-facing

5 Logging information exposure

Logs are a loot pile *and* a cover-up target. Verbose logging captures secrets; insecure log storage leaks them; attackers may also tamper/delete logs to hide.

PICTURE IT Security cameras that accidentally record everyone typing their PIN — and whose footage is left on a public shelf. Sometimes the burglar also wipes the tapes.

FINDS Session tokens/API keys (replayable) · creds in plaintext form submissions · internal IPs & service names · stack traces. Storage sins: **CloudTrail logs in a public S3 bucket**, world-readable log dirs, unencrypted log transmission.

EXAM TELL "attacker accesses log files, what could be retrieved," log tampering to cover tracks, **immutable/log integrity** as the fix → **logging info exposure**.

④ Supply / artifact plane

3 Third-party integrations

Every integration is a **trust relationship** that bypasses your perimeter: insecure APIs, vulnerable libraries, unauthenticated webhooks, over-broad OAuth, and high-privilege CI/CD.

PICTURE IT Contractors with their own key to your building. You vetted your own staff — but the alarm company's intern also has a key.

EXAM TELL External service has access to cloud resources; forged **webhook**; OAuth token theft/replay; compromised **CI/CD pipeline** (deployment perms) → **third-party integration risk**. This is a *runtime* trust risk.

6 Image & artifact tampering

Modify the deployable artifact (VM image/AMI, container image, package) **after it exists** → every deployment from it is compromised, and it scales automatically with auto-scaling / orchestration.

PICTURE IT Slipping a bad ingredient into the master recipe at the factory. Every meal shipped is poisoned, and each one looks perfectly normal.

KEY IDEA **Signed ≠ safe.** Code signing proves *who* signed it, not that the code is benign. Compromise the signing env and malicious code is legitimately signed.

EXAM TELL Malicious code introduced via a **container registry** / tampered AMI / malicious layer, "registry write access," image scanning (Trivy Grype) → **image/artifact tampering**.

7 Supply chain attacks

Compromise a **trusted upstream source** — a library, an update mechanism, a package registry, a build system — to reach many downstream targets at once.

PICTURE IT Poisoning the town's water supply upstream instead of breaking into each house. One source, everyone downstream affected.

FLAVORS Dependency compromise · **typosquatting** (npm/pip/Maven/NuGet) · update-pipeline abuse (**SolarWinds**) · build-system compromise (source review won't catch it) · trusted upstream provider breach.

EXAM TELL Malicious code delivered through an **update** or a **compromised library**, "SolarWinds," typosquat → **supply chain**. Detection: SCA — TruffleHog Grype Trivy OWASP Dependency-Check.

⑤ Runtime plane

8 Workload runtime attacks

Attack the workload **while it executes** — app vulns (SQLi, command injection, deserialization, auth bypass), unpatched OS/runtime, or loose security settings → code execution *inside* the workload.

PICTURE IT Picking the lock on a running machine while it hums along. You're inside the app, not yet outside the container.

CLOUD TWIST The workload usually has **IAM credentials attached** — so runtime compromise instantly becomes a cloud-API foothold. Serverless (Lambda/Azure Functions) is attacked via triggers, input, and over-permissive function roles.

EXAM TELL App vulnerability exploited "what's the next step / impact in a cloud context" → workload runtime; the tell is **code execution inside a running workload** that still holds IAM creds.

9 Container escape

Break the container isolation boundary to reach the **host** and every other container on it. Isolation = namespaces + cgroups + capabilities; misconfigure them and you break out.

PICTURE IT A cruise-ship cabin whose door was left unlocked to the engine room — suddenly you control the whole ship, not just your bunk.

ESCAPE VECTORS **Privileged container** (near-root on host — trivial escape) · **Docker socket** mounted (`/var/run/docker.sock`) · dangerous **volume mounts** (`/etc`, `/proc`) · kernel exploits (shared kernel). K8s: **hostPID**, **hostNetwork**, API-server access, over-scoped service-account tokens.

AFTER ESCAPE Host IAM creds, all co-located containers, host network, node/cluster creds.

EXAM TELL Container config named (privileged / docker.sock / hostPID) + "can it escape?" → **container escape**. Detection: Docker Bench Kube-hunter. (a.k.a. container breakout.)

MNEMONIC VAULT

169 = "I Sixty-nine my creds"

169.254.169.254 = metadata endpoint. Link-local, inside-only, hands out **temporary credentials**. Any time you see this IP, the answer is metadata/SSRF.

STAR = star (*) = admin

Scope **T**oo **A**ll **R**oles. A wildcard `"*"` in a policy is the IAM-misconfig flag. Star = keys to everything.

"PPD" escapes the box

Privileged container · **P**ath mounts (docker.sock, /etc, /proc) · **D**angerous K8s flags (hostPID/hostNetwork). The three container-escape doors.

TUBS = supply chain

Typosquat · **U**pdate mechanism (SolarWinds) · **B**uild system · **S**CA to detect. Upstream trust, poisoned at the source.

"Pacu Pops, Prowler Peeks"

Pacu = active AWS *exploitation* (pops the box).
Prowler = passive AWS *audit* (just peeks). Both AWS-only.

"Scout the whole scene"

ScoutSuite = the *multi-cloud* scout (AWS+Azure+GCP), API-based, passive. When the scenario spans clouds → ScoutSuite.

CONFUSABLES CRAM — THE PAIRS THE EXAM BLURS

Metadata attack	vs		**IAM misconfiguration**
The **delivery mechanism**: SSRF → metadata endpoint → grab creds.	**is**	The **condition** that makes those creds dangerous — the role is over-permissioned. One delivers; the other amplifies.	
Workload runtime attack	vs		**Container escape**
Attacking the **app inside** the container — you get code exec, still boxed in.	**then**	Breaking **out** of the container to the **host**. Runtime gets you in; escape gets you out.	
Supply chain attack	vs		**Image / artifact tampering**
Compromise **earlier** — during development / build / distribution. Affects the **source material**.	**after**	Modify the artifact **after** it's been created. Affects the **deployed artifact**.	
Pacu / Prowler (AWS)	vs		**ScoutSuite (multi-cloud)**
AWS-only. Pacu = active exploitation (needs creds); Prowler = passive CIS/compliance audit (read-only).	**scope**	**Multi-cloud** (AWS/Azure/GCP), API-based, passive config audit. Pick this whenever >1 provider is in scope.	
Docker Bench	vs		**Kube-hunter**
Audits a **Docker host/container** against the CIS Docker Benchmark (PASS/WARN/INFO). Passive.	**layer**	Hunts a **Kubernetes cluster** (API server, etcd, dashboard, RBAC). Remote/internal/network modes; active or passive.	

PART TWO — TOOLS (SCOPE × FUNCTION IS WHAT'S TESTED)

On the exam the tool question is almost always "which one, given the scope?" Nail two axes for each: **which cloud** and **audit vs. exploit**.

TOOL	SCOPE	FUNCTION	ACTIVE / PASSIVE	ONE-LINE TELL
Pacu	AWS only	Exploitation framework ("Metasploit for AWS"): privesc, IAM enum, S3 access, backdoors, disable CloudTrail/GuardDuty	Active	AWS + exploitation, **needs creds** (post-acquisition)
Prowler	AWS only	Config audit vs CIS / GDPR / HIPAA; read-only	Passive	AWS + **audit/compliance**, read-only
ScoutSuite	Multi-cloud (AWS/Azure/GCP)	API-based config audit; HTML report; configurable rulesets	Passive	**Multi-cloud**, no active scanning, detection-

TOOL	SCOPE	FUNCTION	ACTIVE / PASSIVE	ONE-LINE TELL
				safe
Docker Bench	Any Docker host	CIS Docker Benchmark check: host, daemon, images, runtime	Passive	Docker **misconfig** (privileged / socket / root)
Kube-hunter	Any Kubernetes	Finds exposed API server, etcd, dashboard, bad RBAC, privileged pods	Both	K8s assessment; active "hunter" mode exploits to confirm
Cloud-native	Provider-specific	CloudTrail (API log), GuardDuty (threat detect), Config (compliance), Security Hub, IAM Access Analyzer; Azure Defender for Cloud; GCP Security Command Center	Both	Deep integration; **GuardDuty may detect YOUR test**

Native-tool function map (memorize the AWS trio): `CloudTrail` = records every API call · `GuardDuty` = threat detection (analyzes CloudTrail + VPC Flow + DNS) · `AWS Config` = configuration compliance / drift. If a question asks "which native tool logs API activity / detects threats / tracks config," it's these three in that order.

ATTACK CHAINS — THE MULTI-STEP SCENARIOS

Scenario questions walk you across planes. Recognize the whole path so you can answer "what's the next step."

Chain 1 — SSRF to full compromise

SSRF in web app → **Metadata** → temp IAM creds → **IAM misconfig** (role too broad) →
Pacu finds privesc path → **Exposed bucket** accessed → **Log data** yields more creds

Chain 2 — Container to cloud

App vuln → code exec in container → **Container escape** (privileged / docker.sock) →
Host IAM creds via metadata → **IAM misconfig** (instance role = admin) →
ScoutSuite/Pacu enumerate env → **Trust abuse** to other accounts

Chain 3 — Supply chain to persistence

Supply chain hits registry image → **Artifact tampering** plants backdoor →
Runtime auto-deploys bad image → **Workload IAM creds** reused → **Logging exposure** to cover tracks

SCENARIO RED-FLAGS DECODER

See the phrase in the stem → jump to the answer. This is the fastest-scoring table in the guide.

IF THE SCENARIO DROPS…	…THE ANSWER IS
`169.254.169.254` / “SSRF retrieves credentials” / IMDSv1 vs v2	**Metadata service attack**
Wildcard `"*"` policy / key in a public repo / “more permissions than expected”	**IAM misconfiguration** (→ privilege escalation)
Attacker hops from one account/service to another; Kerberoasting; SAML; cross-account role	**Trust relationship abuse** (map with BloodHound)
`0.0.0.0/0` on port 22/3389/3306; over-permissive security group	**Network controls** misconfiguration
“Flat network,” everything in one subnet, easy lateral movement	**Network segmentation** failure
Public S3/Blob bucket; `aws s3 ls`; unauthenticated data access	**Exposed storage bucket**
Kubernetes API / database / dashboard reachable from the internet	**Public access to services**
Tokens/keys found in log files; CloudTrail logs in a public bucket; logs wiped	**Logging information exposure**
Forged webhook; over-broad OAuth token; compromised CI/CD pipeline	**Third-party integration** risk
Malicious code placed in a registry image / tampered AMI; “signed but malicious”	**Image / artifact tampering**
Malicious library / poisoned update / typosquat / SolarWinds	**Supply chain attack**
App vuln → code exec, workload still holds IAM creds; Lambda trigger abuse	**Workload runtime attack**
Privileged container / `/var/run/docker.sock` / hostPID → reach host	**Container escape**
“Which tool exploits AWS?” (needs creds)	**Pacu**
“Which tool audits AWS vs CIS/compliance, read-only?”	**Prowler**
“Which tool assesses multiple clouds?”	**ScoutSuite**

IF THE SCENARIO DROPS…	…THE ANSWER IS
“Assess Docker security config” / “assess a Kubernetes cluster”	**Docker Bench** / **Kube-hunter**

NIGHT-BEFORE CHEAT STRIP

The one chain: SSRF → 169.254.169.254 → temporary IAM creds → cloud API access.

Metadata IP: 169.254.169.254, link-local, inside-only. IMDSv1 = GET, IMDSv2 = PUT-for-token first.

IAM misconfig = fastest privesc. Wildcard `"*"`, stale keys, keys in repos, broad cross-account trust.

Metadata vs IAM: metadata = delivery; IAM = why it hurts.

Runtime vs escape: runtime = inside the app; escape = out to the host.

Supply chain vs tampering: supply chain = before/at build (source); tampering = after (artifact).

Container escape doors (PPD): Privileged, docker.sock/dangerous mounts, hostPID/hostNetwork.

Signed ≠ safe: signing proves who, not that it’s benign.

Pacu = AWS exploit (active, needs creds). **Prowler** = AWS audit (passive, read-only).

ScoutSuite = multi-cloud audit (API, passive). Cross-provider scenario → ScoutSuite.

Docker Bench = Docker CIS check. **Kube-hunter** = K8s (API server, etcd, RBAC).

AWS native trio: CloudTrail = API log · GuardDuty = threat detect · Config = compliance. GuardDuty can flag your test.

0.0.0.0/0 on 22/3389/DB ports = network-control misconfig. **Flat network** = segmentation failure.

Public bucket = zero-exploit data breach; only need the name.

Theme of the whole domain: misconfiguration is the #1 cloud attack vector.

SCA detectors: TruffleHog, Grype, Trivy, OWASP Dependency-Check (supply chain / secrets in images).